

Assignment Questions

Vayu Air · Session 3: Data Modeling & Warehouse Engineering

Before you start

Note Engine is **SQL Server (T-SQL)**, run in SSMS. You are given raw `bronze_*` tables and you will build a dw warehouse. Use `booking_status = 'Confirmed'` for revenue unless a task says otherwise. Estimated effort is 1 to 2 evenings.

Design

1. State the **grain** of the sales fact you will build, written as a comment, and classify every candidate column from `bronze_bookings` (joined to its flight) as a **dimension key** or a **measure**. Add one sentence naming which measures are additive and giving one example of a value that would not be additive. Acceptance criteria: a clear one-row grain statement; a correct split of columns into dimension keys versus measures; one additive-versus-non-additive note.
2. Create a dw schema and write the **star schema DDL**: a central `FactTicketSales` at your declared grain plus the dimensions it needs (date, passenger, flight, airport, aircraft). Use **surrogate keys** on the dimensions and foreign keys on the fact, and keep only additive measures on the fact. Acceptance criteria: fact at the declared grain with surrogate-key foreign keys; each dimension has a surrogate primary key and keeps its business key.

Build

3. **Load** the dimensions and the fact from the `bronze_*` tables, assigning the surrogate keys. Build a `DimDate` with a `yyyymmdd` integer `DateKey`. Populate the fact by joining bookings to the dimensions on their business keys. Acceptance criteria: dimensions and fact are populated; the fact row count matches `bronze_bookings`; every fact row resolves to all of its dimensions.
4. **Snowflake the geography**. Normalise the airport dimension into three linked tables: airport, city, and country. Load them, and write one sentence on the trade-off you are making. Acceptance criteria: three linked tables with foreign keys; a query can resolve an airport all the way up to its country.
5. Rebuild the passenger dimension as **Slowly Changing Dimension Type 2** (a new surrogate key per version, the stable business key, `is_current`, `effective_from`, `effective_to`). Apply `stg_passenger_updates` so that a changed tier or home airport **expires the old version and opens a new one**, and brand-new passengers are inserted.

Acceptance criteria: the two-step expire-then-insert pattern is used; a passenger who changed shows two versions with correct `is_current` and effective dates; new passengers appear as a single current row.

6. The sales fact is the biggest table in the warehouse. **Partition it by date** using a partition function and a partition scheme. Then run two queries: one that filters on the partition key, and one that filters on a non-partition column. Acceptance criteria: a partition function and scheme exist and the fact sits on the scheme; you show, using the actual execution plan, that the first query prunes to a small number of partitions and the second does not, and you explain why.

Curate

7. Two written deliverables.

(a) **Map every table** in this pipeline to a medallion layer (bronze, silver, or gold) and say in a line why each sits where. (b) Write a **data contract** for the `bronze_bookings` feed. Acceptance criteria: every table is assigned a layer with a short reason; the contract states the schema and types, the allowed values for `fare_class` and `booking_status`, a freshness or delivery SLA, the owner, and one example each of a breaking and a non-breaking change.